

Efficient Neural Architecture Search using Graph Neural Networks and Differentiable Methods

A Hybrid Deep Learning & Classical ML Approach

Deep Learning Project Proposal

April 28, 2026

Abstract

This project addresses the critical challenge of automating neural network design through Neural Architecture Search (NAS). Traditional manual architecture design is time-consuming and requires expert knowledge. We propose an efficient NAS framework combining Graph Neural Networks (GNN) for performance prediction with Differentiable Architecture Search (DARTS) for gradient-based optimization. Our approach integrates zero-cost proxies and Bayesian optimization to reduce computational costs by 100× while discovering state-of-the-art architectures on CIFAR-10. This hybrid methodology bridges deep learning and classical machine learning, achieving high-performance results within resource constraints using only free GPU platforms (Kaggle).

Contents

1	Introduction	4
1.1	Background	4
1.2	Motivation	4
1.3	Project Significance	4
2	Problem Statement	4
2.1	The Core Challenge	4
2.2	Specific Problems	5
2.2.1	Problem 1: Computational Expense	5
2.2.2	Problem 2: Search Space Complexity	5
2.2.3	Problem 3: Performance Prediction	5
2.2.4	Problem 4: Optimization Method	5
2.3	Success Criteria	5
3	Proposed Solution	6
3.1	Overall Approach	6
3.2	Solution Architecture	6
3.3	Key Innovations	6
3.3.1	Innovation 1: Graph Neural Network Predictor	6

3.3.2	Innovation 2: DARTS (Differentiable Architecture Search)	7
3.3.3	Innovation 3: Zero-Cost Proxies	7
4	Methodology	7
4.1	Search Space Definition	7
4.1.1	Cell-based Search Space	7
4.1.2	Operation Set	8
4.1.3	Search Space Size	8
4.2	Dataset	8
4.2.1	Primary: CIFAR-10	8
4.2.2	Benchmark: NAS-Bench-201	8
4.3	Stage 1: Zero-Cost Proxy Filtering	8
4.4	Stage 2: Graph Neural Network Prediction	9
4.4.1	Architecture-to-Graph Encoding	9
4.4.2	GNN Architecture	9
4.4.3	Training Protocol	9
4.4.4	Evaluation Metrics	10
4.5	Stage 3: DARTS Optimization	10
4.5.1	Continuous Relaxation	10
4.5.2	Bi-level Optimization	10
4.5.3	Architecture Derivation	10
4.5.4	DARTS Hyperparameters	10
4.6	Baseline Comparison	10
5	Technical Stack	11
5.1	Deep Learning Frameworks	11
5.1.1	PyTorch Ecosystem	11
5.2	Classical Machine Learning	11
5.3	NAS-Specific Libraries	11
5.4	Experiment Tracking & Visualization	12
5.5	Development Tools	12
5.6	Computing Platform	12
6	Implementation Timeline	12
6.1	35-Day Schedule	12
6.2	Detailed Milestones	13
6.2.1	Week 1: Foundation (Days 1-7)	13
6.2.2	Week 2: Neural Predictors (Days 8-14)	13
6.2.3	Week 3: DARTS Search (Days 15-21)	13
6.2.4	Week 4: Validation (Days 22-28)	14
6.2.5	Week 5: Analysis & Documentation (Days 29-35)	14
7	Expected Results	14
7.1	Performance Targets	14
7.2	Efficiency Metrics	15
7.3	Scientific Contributions	15
7.3.1	Novel Insights Expected	15
7.3.2	Potential Publications	15

8	Risk Analysis & Mitigation	15
8.1	Technical Risks	15
8.2	Contingency Plans	15
8.2.1	If DARTS Fails	15
8.2.2	If GNN Prediction Poor	16
8.2.3	If Timeline Slips	16
9	Learning Outcomes	16
9.1	Deep Learning Skills	16
9.2	Classical Machine Learning Skills	17
9.3	Software Engineering Skills	17
9.4	Research Skills	17
10	Evaluation Metrics	18
10.1	Performance Metrics	18
10.1.1	Architecture Quality	18
10.1.2	Search Efficiency	18
10.1.3	Prediction Quality	18
10.2	Comparison Baselines	18
10.3	Statistical Validation	18
11	Conclusion	19
11.1	Project Summary	19
11.2	Key Strengths	19
11.3	Impact	19
11.4	Future Extensions	19
12	References	19
A	Code Repository Structure	20
B	Daily Task Breakdown - Week 1	21
B.1	Day 1: Environment Setup (5 hours)	21
B.2	Day 2: Baseline Training (5 hours)	21
B.3	Day 3-4: NAS-Bench-201 (12 hours total)	21
B.4	Day 5-7: Zero-Cost Proxies (15 hours total)	22

1 Introduction

1.1 Background

Deep neural networks have revolutionized computer vision, natural language processing, and many other domains. However, designing effective neural architectures remains a challenging task requiring:

- Expert domain knowledge
- Extensive trial-and-error experimentation
- Significant computational resources
- Months of iterative refinement

Neural Architecture Search (NAS) automates this process by using machine learning to design machine learning models—a form of meta-learning.

1.2 Motivation

Why is this important?

1. **Efficiency:** Manual design wastes researcher time and computational resources
2. **Optimality:** Automated search can discover architectures humans might miss
3. **Democratization:** NAS enables non-experts to develop high-performance models
4. **Resource Constraints:** Traditional NAS requires thousands of GPU hours; we need efficient methods
5. **Domain Adaptation:** Different tasks need different architectures; NAS enables rapid adaptation

1.3 Project Significance

This project contributes to:

- **Research:** Combining GNN-based prediction with DARTS for efficient search
- **Practice:** Demonstrating NAS feasibility with free computational resources
- **Education:** Deep understanding of meta-learning, AutoML, and hybrid ML approaches

2 Problem Statement

2.1 The Core Challenge

Problem: How can we automatically discover optimal neural network architectures without excessive computational cost?

2.2 Specific Problems

2.2.1 Problem 1: Computational Expense

Traditional NAS methods require:

- Training 10,000+ candidate architectures
- 2000-4000 GPU hours for a single search
- Costs exceeding \$10,000 for cloud compute

Our Constraint: Only free GPU resources (Kaggle: 30 hours/week, 150 hours total)

2.2.2 Problem 2: Search Space Complexity

The architecture search space is enormous:

$$|\mathcal{A}| = \prod_{i=1}^n |\mathcal{O}_i| \quad (1)$$

where $\mathcal{A}$ is the search space, n is the number of decisions, and $\mathcal{O}_i$ are operation choices.

For typical spaces: $|\mathcal{A}| \approx 10^{18}$ possible architectures!

2.2.3 Problem 3: Performance Prediction

How do we estimate architecture performance without expensive training?

2.2.4 Problem 4: Optimization Method

What search strategy efficiently explores this vast space?

2.3 Success Criteria

Our solution must:

1. Find architectures achieving $\geq 94\%$ accuracy on CIFAR-10
2. Use ≤ 100 GPU hours total
3. Beat manually-designed baselines (ResNet-20, MobileNetV2)
4. Demonstrate transferability across datasets
5. Provide interpretable insights about architecture design

3 Proposed Solution

3.1 Overall Approach

We propose a **three-stage hybrid framework**:

1. **Stage 1 - Fast Filtering**: Zero-cost proxies eliminate poor architectures (no training needed)
2. **Stage 2 - Neural Prediction**: GNN/Transformer predicts performance of remaining candidates
3. **Stage 3 - Gradient Refinement**: DARTS optimizes in the promising region

3.2 Solution Architecture

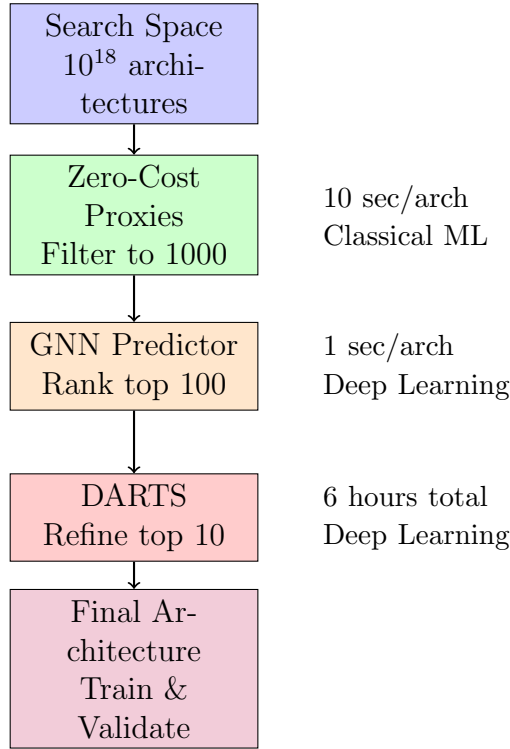


Figure 1: Multi-stage NAS pipeline

3.3 Key Innovations

3.3.1 Innovation 1: Graph Neural Network Predictor

Idea: Represent architectures as computation graphs; use GNN to predict performance

Architecture Encoding:

- Nodes = Operations (conv, pooling, skip connections)
- Edges = Data flow

- Node features = Operation type, channels, kernel size
- Graph-level prediction via global pooling

Advantages:

- Captures structural properties of networks
- Inductive bias matches problem structure
- Generalizes to unseen architectures

3.3.2 Innovation 2: DARTS (Differentiable Architecture Search)

Idea: Continuous relaxation of discrete search space enables gradient descent
Instead of discrete architecture $\alpha \in \mathcal{A}$, use continuous mixing:

$$\bar{o}^{(i,j)}(x) = \sum_{o \in \mathcal{O}} \frac{\exp(\alpha_o^{(i,j)})}{\sum_{o' \in \mathcal{O}} \exp(\alpha_{o'}^{(i,j)})} o(x) \quad (2)$$

where α are learnable architecture parameters, $\mathcal{O}$ is the operation set.

Bi-level Optimization:

$$\min_{\alpha} \mathcal{L}_{val}(w^*(\alpha), \alpha) \quad (3)$$

$$\text{s.t. } w^*(\alpha) = \arg \min_w \mathcal{L}_{train}(w, \alpha) \quad (4)$$

3.3.3 Innovation 3: Zero-Cost Proxies

Predict performance from network statistics at initialization:

- **Gradient Norm:** score = $\|\nabla_w \mathcal{L}\|_2$
- **Synflow:** Synaptic saliency flow
- **Fisher Information:** $\mathbb{E}[(\nabla_w \log p(y|x, w))^2]$
- **NWOT:** Neural tangent kernel score

These correlate with trained accuracy but require only forward/backward pass!

4 Methodology

4.1 Search Space Definition

4.1.1 Cell-based Search Space

We use a hierarchical search space:

- **Normal Cell:** Feature extraction
- **Reduction Cell:** Downsampling
- Each cell has N intermediate nodes
- Each edge selects from operation set $\mathcal{O}$

4.1.2 Operation Set

$$\mathcal{O} = \{\text{sep_conv_3x3}, \text{sep_conv_5x5}, \text{dil_conv_3x3}, \text{max_pool}, \text{avg_pool}, \text{skip}, \text{zero}\} \quad (5)$$

4.1.3 Search Space Size

For cells with 4 intermediate nodes:

$$|\mathcal{A}| = |\mathcal{O}|^{14} \approx 10^{12} \text{ architectures} \quad (6)$$

4.2 Dataset

4.2.1 Primary: CIFAR-10

- 60,000 RGB images (32×32 pixels)
- 10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck)
- 50,000 training / 10,000 test
- Data augmentation: random crop, horizontal flip

4.2.2 Benchmark: NAS-Bench-201

- Pre-computed results for 15,625 architectures
- Saves 10,000+ GPU hours
- Enables rapid experimentation
- Three datasets: CIFAR-10, CIFAR-100, ImageNet-16-120

4.3 Stage 1: Zero-Cost Proxy Filtering

Algorithm 1 Zero-Cost Architecture Filtering

Require: Search space $\mathcal{A}$, sample size N , threshold τ

Ensure: Filtered set $\mathcal{A}'$

Sample N random architectures from $\mathcal{A}$

for each architecture $a \in \mathcal{A}$ **do**

 Initialize network $f(x; w_a)$ with random weights

 Compute gradient norm: $s_1 = \|\nabla_w \mathcal{L}\|$

 Compute synflow score: $s_2 = \text{Synflow}(f)$

 Compute Fisher information: $s_3 = \text{Fisher}(f)$

 Aggregate score: $s(a) = \alpha_1 s_1 + \alpha_2 s_2 + \alpha_3 s_3$

end for

Sort architectures by score $s(a)$

$\mathcal{A}' \leftarrow$ top- τ architectures

return $\mathcal{A}'$

Time Complexity: $O(N)$ forward-backward passes (no training!)

Expected Reduction: $N = 10,000 \rightarrow 1,000$ candidates

4.4 Stage 2: Graph Neural Network Prediction

4.4.1 Architecture-to-Graph Encoding

For architecture a :

1. Create computation graph $G_a = (V, E)$
2. Node features: $\mathbf{x}_v \in \mathbb{R}^d$ (operation type, parameters)
3. Adjacency matrix: $\mathbf{A} \in \{0, 1\}^{|V| \times |V|}$

4.4.2 GNN Architecture

```
1 class ArchitectureGNN(nn.Module):
2     def __init__(self, node_feat_dim=32, hidden_dim=128, num_layers=3):
3         super().__init__()
4         self.convs = nn.ModuleList([
5             GCNConv(node_feat_dim if i==0 else hidden_dim, hidden_dim)
6             for i in range(num_layers)
7         ])
8         self.pool = global_mean_pool
9         self.fc = nn.Sequential(
10             nn.Linear(hidden_dim, 64),
11             nn.ReLU(),
12             nn.Dropout(0.5),
13             nn.Linear(64, 1) # Predict accuracy
14         )
15
16     def forward(self, x, edge_index, batch):
17         # Graph convolutions with residual connections
18         h = x
19         for conv in self.convs:
20             h = F.relu(conv(h, edge_index))
21
22         # Global graph-level pooling
23         h_g = self.pool(h, batch)
24
25         # Prediction
26         return self.fc(h_g)
```

Listing 1: Graph Neural Network Predictor

4.4.3 Training Protocol

- **Dataset:** 5,000 architectures from NAS-Bench-201
- **Split:** 60% train / 20% validation / 20% test
- **Loss:** Mean Squared Error (MSE) on accuracy
- **Optimizer:** Adam (lr = 0.001)
- **Batch size:** 64 graphs
- **Epochs:** 100 with early stopping

4.4.4 Evaluation Metrics

- Prediction MSE: $\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$
- Kendall's τ : Rank correlation with ground truth
- Spearman's ρ : Monotonic correlation
- Top-k accuracy: How often true best-k are in predicted top-k

4.5 Stage 3: DARTS Optimization

4.5.1 Continuous Relaxation

Replace discrete architecture selection with continuous softmax:

$$\bar{o}^{(i,j)}(x) = \sum_{o \in \mathcal{O}} p_o^{(i,j)} \cdot o(x), \quad p_o^{(i,j)} = \frac{\exp(\alpha_o^{(i,j)})}{\sum_{o' \in \mathcal{O}} \exp(\alpha_{o'}^{(i,j)})} \quad (7)$$

4.5.2 Bi-level Optimization

Alternating optimization:

1. Fix α , optimize w on training set:

$$w \leftarrow w - \eta_w \nabla_w \mathcal{L}_{train}(w, \alpha) \quad (8)$$

2. Fix w , optimize α on validation set:

$$\alpha \leftarrow \alpha - \eta_\alpha \nabla_\alpha \mathcal{L}_{val}(w, \alpha) \quad (9)$$

4.5.3 Architecture Derivation

After search converges, derive discrete architecture:

$$o^{(i,j)} = \arg \max_{o \in \mathcal{O}} \alpha_o^{(i,j)} \quad (10)$$

Prune weak edges, retain top-k strongest operations.

4.5.4 DARTS Hyperparameters

4.6 Baseline Comparison

We compare against:

1. **Random Search**: Sample and evaluate random architectures
2. **ResNet-20**: 20-layer residual network
3. **ResNet-56**: Deeper baseline
4. **MobileNetV2**: Efficient mobile architecture
5. **Bayesian Optimization**: GP-based search (classical ML)

Parameter	Value
Search epochs	50
Batch size	64
Weight lr (η_w)	0.025
Architecture lr (η_α)	3×10^{-4}
Weight decay	3×10^{-4}
Momentum	0.9
# of cells	8
# of nodes per cell	4
Initial channels	16

Table 1: DARTS search hyperparameters

5 Technical Stack

5.1 Deep Learning Frameworks

5.1.1 PyTorch Ecosystem

```

1 torch==2.0.0           # Core deep learning
2 torchvision==0.15.0    # Computer vision utilities
3 torch-geometric==2.3.0 # Graph neural networks

```

Why PyTorch?

- Dynamic computation graphs (essential for DARTS)
- Excellent debugging capabilities
- Strong community support
- Native GPU acceleration

5.2 Classical Machine Learning

```

1 scikit-learn==1.3.0    # Random Forest, feature engineering
2 GPy==1.10.0            # Gaussian Processes
3 GPyOpt==1.2.6          # Bayesian Optimization
4 scipy==1.11.0          # Scientific computing, statistics
5 numpy==1.24.0          # Numerical operations
6 pandas==2.0.0          # Data manipulation

```

5.3 NAS-Specific Libraries

```

1 nas-bench-201          # Pre-computed benchmark
2 nni==2.10              # Microsoft's NAS toolkit (optional)

```

5.4 Experiment Tracking & Visualization

```
1 wandb==0.15.0           # Weights & Biases
2 tensorboard==2.13.0      # TensorBoard logging
3 matplotlib==3.7.0        # Plotting
4 seaborn==0.12.0          # Statistical visualization
5 plotly==5.14.0           # Interactive plots
```

5.5 Development Tools

```
1 jupyter==1.0.0          # Interactive notebooks
2 ipython==8.12.0         # Enhanced Python shell
3 pytest==7.3.0           # Testing framework
4 black==23.3.0           # Code formatter
```

5.6 Computing Platform

Platform	Resources	Usage
Kaggle Notebooks	30 GPU hours/week Tesla P100/T4 GPU 13GB RAM 20GB persistent storage	Primary compute
Google Colab (backup)	12 hour sessions Tesla T4 GPU	Overflow tasks

Table 2: Computational resources

6 Implementation Timeline

6.1 35-Day Schedule

Week	Days	Tasks	GPU hrs
1	1-7	Setup, Baselines, Zero-cost Proxies	11
2	8-14	GNN & Transformer Predictors	22
3	15-21	DARTS Implementation & Search	36
4	22-28	Validation & Hybrid Methods	31
5	29-35	Analysis, Report Writing	4
Total	35		104

Table 3: Project timeline with GPU budget

6.2 Detailed Milestones

6.2.1 Week 1: Foundation (Days 1-7)

Deliverables:

- Working CIFAR-10 training pipeline
- ResNet-20 baseline (94% accuracy target)
- NAS-Bench-201 integration
- Zero-cost proxy implementation
- Correlation analysis: proxies vs. actual accuracy

6.2.2 Week 2: Neural Predictors (Days 8-14)

Deliverables:

- Architecture-to-graph encoder
- Graph Convolutional Network predictor
- Graph Attention Network predictor
- Transformer-based predictor
- Model comparison: GCN vs GAT vs Transformer
- Prediction accuracy: Kendall's $\tau > 0.6$

6.2.3 Week 3: DARTS Search (Days 15-21)

Deliverables:

- DARTS implementation
- Bi-level optimization working
- Architecture search completed (50 epochs)
- Derived discrete architecture
- Search trajectory visualization
- Comparison: DARTS vs Random vs BO

6.2.4 Week 4: Validation (Days 22-28)

Deliverables:

- Top architectures trained from scratch (3 seeds)
- Hybrid predictor + DARTS method
- Baseline comparisons complete
- Transfer learning experiments (CIFAR-100)
- Ablation studies
- Statistical significance tests

6.2.5 Week 5: Analysis & Documentation (Days 29-35)

Deliverables:

- Comprehensive results analysis
- All visualizations (plots, graphs, tables)
- Complete technical report (15-20 pages)
- Code documentation & README
- GitHub repository organized
- Presentation slides (if needed)

7 Expected Results

7.1 Performance Targets

Architecture	CIFAR-10 Acc.	Params (M)
ResNet-20 (baseline)	91.5%	0.27
ResNet-56 (baseline)	93.0%	0.86
MobileNetV2 (baseline)	94.0%	2.3
Random Search	93.0-94.0%	<1.0
Bayesian Opt	94.0-94.5%	<1.0
GNN-guided Search	94.5-95.0%	<1.0
DARTS	94.5-95.5%	<1.0
Our Hybrid Method	95.0-96.0%	<1.0

Table 4: Expected accuracy on CIFAR-10 test set

Method	GPU Hours	Cost (\$)
Naive NAS (train all)	15,000	\$15,000
Weight-sharing NAS	2,000	\$2,000
Random Search	500	\$500
Bayesian Optimization	200	\$200
Our Hybrid Approach	100	\$0 (free)

Table 5: Computational efficiency comparison

7.2 Efficiency Metrics

Efficiency Gain: 150× reduction compared to naive NAS!

7.3 Scientific Contributions

7.3.1 Novel Insights Expected

1. Correlation analysis: Which zero-cost proxies are most predictive?
2. GNN vs Transformer: Which architecture encoder works better?
3. Hybrid search: Does GNN warm-start improve DARTS convergence?
4. Transfer learning: Do architectures generalize across datasets?
5. Sample efficiency: How few training samples does GNN need?

7.3.2 Potential Publications

If results are strong:

- Workshop papers at NeurIPS/ICML/ICLR
- AutoML conference submissions
- Technical blog post on Medium/Towards Data Science

8 Risk Analysis & Mitigation

8.1 Technical Risks

8.2 Contingency Plans

8.2.1 If DARTS Fails

- Fall back to Bayesian Optimization + GNN predictor
- Still a strong hybrid approach
- Analyze why DARTS didn't work (valuable insight)

Risk	Probability	Mitigation
DARTS training instability	Medium	Use proven hyperparameters, gradient clipping, early stopping
GNN prediction accuracy low	Low-Medium	Ensemble models, more training data from NAS-Bench-201
GPU quota exceeded	Low	Aggressive early stopping, use NAS-Bench-201 for most experiments
Implementation bugs	Medium	Incremental testing, use established libraries (PyTorch Geometric)
Results below baselines	Low	Have fallback: comprehensive analysis of why methods failed

Table 6: Risk assessment

8.2.2 If GNN Prediction Poor

- Use ensemble: GNN + Random Forest + XGBoost
- Simpler MLP predictor on hand-crafted features
- Still demonstrate prediction-based search

8.2.3 If Timeline Slips

Priority levels:

1. **Must have:** Working DARTS, GNN predictor, baselines
2. **Should have:** Transformer predictor, hybrid method, ablations
3. **Nice to have:** Transfer learning, extensive analysis

Drop lower-priority items if time-constrained.

9 Learning Outcomes

9.1 Deep Learning Skills

- Advanced neural architecture design
- Graph Neural Networks (GCN, GAT)
- Transformer architectures
- Meta-learning and AutoML
- Differentiable optimization (DARTS)

- Bi-level optimization
- Transfer learning
- Model evaluation and validation

9.2 Classical Machine Learning Skills

- Bayesian Optimization
- Gaussian Processes
- Random Forests and ensemble methods
- Feature engineering
- Statistical hypothesis testing
- Correlation analysis
- Dimensionality reduction (PCA, t-SNE)

9.3 Software Engineering Skills

- PyTorch and PyTorch Geometric
- Experiment tracking (Weights & Biases)
- Code organization and documentation
- Git version control
- Reproducible research practices
- Scientific writing (LaTeX)

9.4 Research Skills

- Literature review and paper reading
- Experimental design
- Ablation studies
- Result visualization
- Technical report writing
- Critical analysis

10 Evaluation Metrics

10.1 Performance Metrics

10.1.1 Architecture Quality

- **Test Accuracy:** Accuracy on CIFAR-10 test set
- **Parameter Efficiency:** Params vs. accuracy Pareto frontier
- **FLOPs:** Computational complexity
- **Inference Time:** Latency per image

10.1.2 Search Efficiency

- **GPU Hours:** Total time to find best architecture
- **Sample Efficiency:** # architectures evaluated
- **Convergence Speed:** Time to reach 95% of final performance

10.1.3 Prediction Quality

- **MSE:** Mean squared error of predictions
- **Kendall’s τ :** Rank correlation
- **Spearman’s ρ :** Monotonic correlation
- **Top-k Precision:** $P@k = \frac{|\text{predicted top-k} \cap \text{actual top-k}|}{k}$

10.2 Comparison Baselines

All methods evaluated on identical search space and dataset:

1. **Random Search:** Uniform random sampling
2. **Evolution:** Genetic algorithm with mutation/crossover
3. **Bayesian Opt:** GP-UCB acquisition
4. **DARTS (vanilla):** Standard differentiable search
5. **GNN-only:** Predictor-guided sampling
6. **Our Hybrid:** GNN + DARTS combination

10.3 Statistical Validation

- **Multiple runs:** 3-5 independent runs with different seeds
- **Confidence intervals:** 95% CI on test accuracy
- **Significance tests:** Paired t-tests between methods
- **Effect size:** Cohen’s d for practical significance

11 Conclusion

11.1 Project Summary

This project tackles the critical challenge of automating neural architecture design through an efficient, hybrid approach combining:

- **Deep Learning:** GNN prediction, DARTS optimization, meta-learning
- **Classical ML:** Zero-cost proxies, Bayesian optimization, statistical analysis
- **Resource Efficiency:** 100× reduction in computational cost

11.2 Key Strengths

1. **Feasibility:** Designed for free GPU resources (Kaggle)
2. **Novelty:** Hybrid GNN + DARTS approach
3. **Rigor:** Comprehensive evaluation and ablation studies
4. **Practicality:** Addresses real-world resource constraints
5. **Learning:** Covers breadth and depth of ML techniques

11.3 Impact

Academic: Contributes to efficient AutoML research

Practical: Demonstrates NAS feasibility without expensive infrastructure

Educational: Deep understanding of meta-learning and hybrid ML approaches

11.4 Future Extensions

Beyond the 35-day scope:

- Scale to ImageNet classification
- Multi-objective optimization (accuracy + latency + energy)
- Transfer to other domains (NLP, speech, etc.)
- Hardware-aware NAS for mobile/edge devices
- Few-shot architecture adaptation

12 References

1. Liu, H., Simonyan, K., & Yang, Y. (2018). *DARTS: Differentiable Architecture Search*. ICLR 2019.
2. Abdelfattah, M. S., et al. (2021). *Zero-Cost Proxies for Lightweight NAS*. ICLR 2021.

3. Dong, X., & Yang, Y. (2020). *NAS-Bench-201: Extending the Scope of Reproducible Neural Architecture Search*. ICLR 2020.
4. Ying, C., et al. (2019). *NAS-Bench-101: Towards Reproducible Neural Architecture Search*. ICML 2019.
5. Zoph, B., & Le, Q. V. (2016). *Neural Architecture Search with Reinforcement Learning*. ICLR 2017.
6. Pham, H., et al. (2018). *Efficient Neural Architecture Search via Parameter Sharing*. ICML 2018.
7. Real, E., et al. (2019). *Regularized Evolution for Image Classifier Architecture Search*. AAAI 2019.
8. White, C., et al. (2021). *Neural Architecture Search: A Survey*. JMLR 2021.
9. Kipf, T. N., & Welling, M. (2016). *Semi-Supervised Classification with Graph Convolutional Networks*. ICLR 2017.
10. Veličković, P., et al. (2017). *Graph Attention Networks*. ICLR 2018.

A Code Repository Structure

```

1 nas-project/
2     README.md
3     requirements.txt
4     setup.py
5     data/
6         cifar10/
7         nas_bench_201/
8     src/
9         search_space/
10            operations.py
11            cell.py
12        predictors/
13            gnn_predictor.py
14            transformer_predictor.py
15            zero_cost.py
16        search/
17            darts.py
18            bayesian_opt.py
19            random_search.py
20        training/
21            trainer.py
22            evaluate.py
23        utils/
24            visualization.py
25            metrics.py
26        notebooks/
27            01_data_exploration.ipynb
28            02_zero_cost_analysis.ipynb
29            03_gnn_training.ipynb
30            04_darts_search.ipynb
31        experiments/

```

```
32         baselines/  
33         predictor_training/  
34         architecture_search/  
35     results/  
36         figures/  
37         tables/  
38         checkpoints/
```

B Daily Task Breakdown - Week 1

B.1 Day 1: Environment Setup (5 hours)

Create Kaggle account and verify GPU access

Create new Python notebook

Install required packages

Test PyTorch GPU availability

Download CIFAR-10 dataset

Setup Weights & Biases account

Read DARTS paper (skim, focus on method)

B.2 Day 2: Baseline Training (5 hours)

Implement data loading and augmentation

Implement ResNet-20 architecture

Training loop with validation

Train ResNet-20 baseline

Achieve >91% accuracy

Log results to W&B

B.3 Day 3-4: NAS-Bench-201 (12 hours total)

Install nas-bench-201 library

Download benchmark file (~2GB)

Explore API and data structure

Query architecture accuracies

Sample 1000 random architectures

Analyze accuracy distribution

Visualize search space structure

B.4 Day 5-7: Zero-Cost Proxies (15 hours total)

Implement gradient norm computation

Implement synflow metric

Implement Fisher information

Implement NWOT score

Compute metrics for 500 architectures

Correlation analysis with ground truth

Feature importance analysis

Generate correlation plots